

Software Requirement Specifications

**CultiKure: An AI-Powered Web Application for Plant
Disease Detection and Cure Recommendation**

Version: 1.0



Submitted by:

Abdullah Javeed

F22BINFT1M01076

Submitted to:

DR. Syed Ali Nawaz Shah

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Meeting Details

Date	Discussion	Participants	Outcome
04-10-2025	Final Year Project idea discussion and problem identification	Student, Supervisor	Problem domain finalized (Agriculture + AI)
18-10-2025	Project title selection and feasibility analysis	Student, Supervisor	Project title “CultiKure: AI-Powered Plant Disease Detection” approved
30-10-2025	Literature review and analysis of similar systems	Student, Supervisor	Literature review approach approved
12-11-2025	Project scope, functional and non-functional requirements	Student, Supervisor	Project scope finalized
05-12-2025	Review of Software Requirements Specification (SRS)	Student, Supervisor	SRS approved with minor suggestions

Summary

Although many parts of Pakistan's economy rely on the success of the agricultural sector, the industry is continuously hampered by the slow and often wrong identification of plant illnesses which results in financial losses due to underperforming crop yields in the country. Slow manual crop inspections by agricultural specialists, which are in short supply, create delays and allow farmers to lose control of the plants. To fill the gap, I propose to build a responsive, centralized web application called "**CultiKure**". In my assessment, I concluded that the most optimal path to a functional system is an "**API-first**" Architecture with a focus on integrating niche custom-built APIs. To that end, the system deploys the Plant.id API for precise image capture and the Google Gemini API for contextually relevant prescription generation. CultiKure Design Specification defines the minimum viable product as an application for use on any device and is complemented by the ability to log onto a secure microservices environment for the application to effect image streaming, distribute control, and orchestrate system activity with AI services. The end goal of CultiKure is to provide an agricultural expert system that is instant and fully automated with workflows that include a prescription that contains an advanced drug or device for the crop.

Table of Contents

1. Introduction	5
1.1. Purpose	5
1.2. Scope.....	5
1.3. Product Perspective.....	6
1.4. User Characteristics	7
1.5. Similar apps and systems/Literature Review	7
1.6. Proposed Technologies	9
2. Requirements:.....	10
2.1. Functional Requirements	10
2.2. Non-Functional Requirements	12
3. Use Cases and Flow of Processes	14
3.1. Use Case 1.....	15
3.2. Use Case 2.....	16
3.3. Use Case 3.....	17
3.4. Use Case 4.....	18
3.5. Use Case 5.....	19
4. References	20

1. Introduction

CultiKure represents a strategic application of software engineering to solve a real-world problem. By democratizing access to disease diagnostics, the platform aims to reduce crop wastage and enhance overall agricultural productivity. This SRS document serves as the blueprint for the successful development of this high-impact tool.

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the requirements for the "CultiKure" web application. This document illustrates the intended purpose, comprehensive scope, functional and non-functional requirements, and system constraints. It serves as a guideline for the development team (developers, testers, and designers) and stakeholders to ensure that the final web application meets the user's needs for accurate plant disease detection and cure recommendation.

1.2. Scope

Project Boundaries The scope of the "CultiKure" project is strictly defined as the development of a responsive **Web Application** to facilitate the automated diagnosis of plant diseases. The project boundaries connect end-users (farmers and gardeners) with advanced third-party AI services through a centralized web interface. The system operates entirely online, requiring active internet connectivity to perform real-time data processing and API communication.

Items to be Developed (In Scope) The following specific modules and functionalities will be developed and delivered as part of this project:

- **Responsive Web Interface:** A user-friendly frontend built with HTML, CSS, and JavaScript that adapts seamlessly to both desktop and mobile web browsers.
- **Backend System:** A server-side application developed using Python (Flask) to manage logic, security, and data flow.
- **Dual-API Integration Module:**
 - Integration with the **Plant.id API** to process user-uploaded images and return disease diagnoses.
 - Integration with the **Google Gemini API** to generate detailed symptoms, causes, and cure recommendations.
- **User Management System:** Functionalities for secure user registration, authentication (login), and profile management.
- **Scan History Dashboard:** A database-driven module to store, retrieve, and display a user's past diagnostic records and treatment advice.

Items Not to be Developed (Out of Scope) The following items are explicitly excluded from the scope of this project and will **not** be developed:

- **Native Mobile Applications:** No standalone Android (.apk) or iOS (.ipa) applications will be created. The system is web-based only.
- **Custom AI Model Training:** The project will not involve collecting datasets or training custom Deep Learning models (CNNs) from scratch. It relies exclusively on pre-trained external APIs.
- **Offline Diagnostics:** The application will not support offline disease detection; an active internet connection is mandatory for core functionality.
- **Hardware Components:** The project does not involve the development or integration of any hardware devices such as agricultural drones or IoT sensors.

1.3. Product Perspective

System Integration and Fit: The "CultiKure" project is designed as a modular, cloud-based web application that fits into a distributed computing environment. Unlike a standalone desktop software that processes data locally, this developed module functions as the central **Integration Hub** within a larger ecosystem of user devices and external artificial intelligence services.

The developed module fits into the system architecture as follows:

1. **User Interface Layer (The Client):** The module operates within the user's existing environment by running on standard web browsers (Google Chrome, Safari, Microsoft Edge). It utilizes the client's hardware (camera and display) to capture inputs and present results, requiring no native installation on the device.
2. **Coordination Layer (The Backend):** The core application logic (developed in Python/Flask) acts as the system controller. It receives data from the user and orchestrates the flow of information to and from external subsystems.
3. **Dependency on External Subsystems:** The module is not isolated; it relies on real-time integration with third-party services to function:
 - **Visual Processing Engine:** The system connects to the **Plant.id API** to perform complex image analysis.
 - **Knowledge Engine:** The system connects to the **Google Gemini API** to generate textual cure recommendations.
4. **Data Persistence Layer:** The module interfaces with a cloud database (Firestore/SQLite) to maintain a persistent record of User Profiles and Scan History, ensuring data availability across different sessions.

Operating Environment Constraints Since the module relies on these external connections, it fits strictly into an **Online Operating Environment**. It requires active internet connectivity to bridge the user's request with the cloud-based AI engines and cannot function as an offline standalone utility. Yes, users can access the history and other functions of application without internet.

1.4. User Characteristics

1. Farmers and Home Gardeners (The "End Users")

- **Role:** These are the primary end-users of the system who seek to identify plant diseases.
- **Characteristics:** This group is diverse, ranging from professional farmers to hobbyist gardeners. Their technical literacy may vary from low to moderate. Therefore, they require a highly intuitive, simple, and responsive User Interface (UI) that requires minimal training.
- **Interactions:** Their interaction is limited to the frontend interface for registering accounts, uploading leaf images, viewing diagnosis results, and reading educational content. They prioritize speed, accuracy, and ease of use on mobile devices.

2. System Administrator (The "Admin")

- **Role:** The super-user responsible for the overall maintenance, security, and operational integrity of the web application.
- **Characteristics:** This user possesses high technical proficiency and understands web technologies.
- **Interactions:** The Admin has full access to the backend system. Their responsibilities include managing user accounts, monitoring API usage quotas (for Plant.id and Google Gemini), ensuring database security, and handling system configurations.

1.5. Similar apps and systems/Literature Review

Literature Review The integration of Artificial Intelligence (AI) in agriculture has transformed traditional farming practices. Historically, disease detection relied on visual inspection by experts, which is often slow, subjective, and prone to human error. Recent literature highlights a shift towards Computer Vision and Deep Learning (DL) techniques, specifically Convolutional Neural Networks (CNNs), to automate this process. However, a significant gap remains in making these advanced technologies accessible to small-scale farmers without requiring high-end hardware or complex software installations.

Existing Systems and Similar Apps Several mobile applications currently exist in the market that address plant disease detection. An analysis of these systems reveals their features and limitations:

1. Plantix - Your Crop Doctor

- **Features:** Plantix is a widely used mobile application that utilizes image recognition to detect plant diseases, pests, and nutrient deficiencies. It offers a "Community" feature where farmers can discuss issues and get weather alerts.
- **Shortcomings:**

- **Platform Dependency:** It is primarily a native Android application, requiring download and installation, which consumes storage space on the user's device.
- **Limited Accessibility:** It is not fully accessible via desktop web browsers for users who prefer a larger screen or do not have a compatible smartphone.
- **Static Responses:** The cure recommendations are often retrieved from a static database, lacking the contextual depth that generative AI can provide.

2. PictureThis - Plant Identifier

- **Features:** This app is popular for identifying plant species and diagnosing common problems. It provides care tips, watering reminders, and a user-friendly interface.
- **Shortcomings:**
 - **Cost Barrier:** The application operates on a "Freemium" model with aggressive monetization. Most advanced diagnostic features require a paid subscription, making it inaccessible for low-income farmers.
 - **Focus Area:** Its primary focus is on ornamental plants and home gardening (botany) rather than agricultural crops and yield improvement.

3. Agrio - Precision Agriculture

- **Features:** Agrio integrates satellite imagery with AI to monitor fields remotely. It provides hyper-local weather forecasts and allows teams to collaborate on scouting reports.
- **Shortcomings:**
 - **Complexity:** The system is designed for large-scale enterprise farming and professional agronomists. Its complex interface and advanced features (like satellite indexing) are often overwhelming for small-scale farmers with limited technical literacy.
 - **Registration Barrier:** Requires a lengthy registration process and team setup before usage.

Gap Analysis (Why CultiKure?) While existing systems offer robust detection capabilities, they often suffer from platform restrictions (mobile-only), high costs (subscriptions), or excessive complexity. **CultiKure** addresses these gaps by offering a **Web-First approach**, eliminating the need for installation. Furthermore, by integrating **Google Gemini**, CultiKure moves beyond simple static advice to provide detailed, AI-generated explanations and treatment plans, democratizing access to expert knowledge for free

1.6. Proposed Technologies

To develop the "CultiKure" web application, a robust and modern technology stack has been selected to ensure performance, scalability, and ease of development. The system is divided into Frontend, Backend, Database, and External AI Services.

1. Frontend Technologies (User Interface)

- **HTML5:** Used for structuring the web pages and content.
- **CSS3:** Used for styling the application and ensuring a **Responsive Design** so that the website adapts seamlessly to both desktop and mobile screens.
- **JavaScript (ES6+):** Used for client-side logic, handling image uploads, and making asynchronous requests (AJAX/Fetch API) to the backend server without reloading the page.

2. Backend Technologies (Server-Side)

- **Python:** The primary programming language selected for its simplicity and rich support for API integration.
- **Flask:** A lightweight Python web framework used to build the RESTful API endpoints, handle user authentication, and manage routing between the frontend and external AI services.

3. Artificial Intelligence & APIs (The Core Engine)

- **Plant.id Health Assessment API:** A specialized third-party computer vision service used for accurately detecting plant diseases from images.
- **Google Gemini API:** A generative AI service used to process the diagnosis result and generate detailed, human-readable information regarding symptoms, causes, and cure recommendations.

4. Database Management

- **SQLite (Development):** A lightweight relational database used during the initial development phase for storing user profiles and history.
- **Firebase Firestore (Production - Optional):** A NoSQL cloud database for scalable real-time data management (to be considered for final deployment).

5. Development Tools

- **Visual Studio Code (VS Code):** The primary Integrated Development Environment (IDE).
- **Postman:** Used for testing API responses before integrating them into the code.
- **Git:** Version control system for tracking code changes.

2. Requirements:

The CultiKure web application functions as an integrated diagnostic platform designed to streamline plant health management for farmers and gardeners. The system operates through several interconnected modules, beginning with a secure User Management interface that allows users to register, log in, and maintain a personalized profile. The core functionality revolves around the Diagnostic module, which enables users to capture or upload high-resolution images of plant leaves via a responsive web interface. These images are processed in real-time by the integrated Plant.id API to accurately detect specific diseases. Upon successful identification, the system automatically engages the Treatment Recommendation function, which queries the Google Gemini API to generate and display comprehensive disease details, including symptoms, underlying causes, and actionable cure plans comprising both organic and chemical remedies. Furthermore, the system includes a History Management function that persistently stores all diagnostic results and images in a database, allowing users to access a log of their previous scans and treatments at any time for ongoing crop monitoring.

2.1. Functional Requirements

The specific functional requirements for the "CultiKure" web application are defined below:

2.1.1 User Registration

- **Name:** FR001
- **Purpose:** To allow new users (Farmers, Gardeners) to create an account to access the dashboard and history features.
- **User(s):** Guest User
- **Input:**
 - **Full Name:** Alphabetic characters representing the user's identity.
 - **Email Address:** A valid email format (e.g., user@example.com) to serve as a unique User ID.
 - **Password:** Must be at least 8 characters long and include alphanumeric characters for security.
 - **Confirm Password:** Must match the entered password exactly.
- **Output:** A new user profile is created in the database, and the user is redirected to the Login page.

2.1.2 User Login

- **Name:** FR002

- **Purpose:** To authenticate registered users and grant them access to the system's core features.
- **User(s):** Registered User, Administrator
- **Input:**
 - **Email Address:** The registered email associated with the account.
 - **Password:** The correct password matching the database record.
- **Output:** The system validates credentials. On success, the user is redirected to the main Dashboard; on failure, an error message is displayed.

2.1.3 Upload Plant Image (Disease Detection)

- **Name:** FR003
- **Purpose:** To enable users to submit a plant leaf image for AI analysis.
- **User(s):** Registered User
- **Input:**
 - **Image File:** A digital file in supported formats (JPG, PNG, JPEG).
 - **File Size Constraint:** Maximum file size of 5MB to ensure efficient API processing.
 - **Source:** Captured via device camera or selected from the gallery.
- **Output:** The image is validated and sent to the **Plant.id API**. The interface shows a "Analyzing..." status.

2.1.4 View Diagnosis and Cure

- **Name:** FR004
- **Purpose:** To display the disease identification results and treatment advice generated by the AI.
- **User(s):** Registered User
- **Input:**
 - **System Trigger:** Automatically triggered upon receiving a response from the Plant.id and Gemini APIs.
- **Output:** The system presents a result card containing:
 - **Disease Name:** (e.g., "Tomato Early Blight").
 - **Detailed Info:** Symptoms and Causes (from Gemini).
 - **Cure Plan:** Organic and Chemical remedies.

2.1.5 View Scan History

- **Name:** FR005
- **Purpose:** To allow users to view a log of their past diagnostic activities.
- **User(s):** Registered User
- **Input:**
 - **View Request:** User clicks on the "History" tab on the dashboard.
- **Output:** A chronological list of previous scans is displayed, showing the plant image thumbnail, detected disease, and date.

2.2. Non-Functional Requirements

Non-functional requirements define the quality attributes, operating constraints, and performance standards of the "CultiKure" web application. While functional requirements describe *what* the system does, these requirements specify *how well* the system must perform its functions to ensure a satisfactory user experience for farmers and gardeners.

1. Performance Requirements

- **Response Time:** The system generally aims to provide the complete diagnosis and cure recommendation within **5 to 10 seconds** of image submission. However, this is contingent upon the latency of the external third-party APIs (Plant.id and Google Gemini) and the user's internet speed.
- **Throughput:** The backend server (Flask) shall be capable of handling concurrent requests from multiple users without crashing, ensuring smooth operation during peak usage times.
- **Image Processing:** The system shall optimize and resize uploaded images (max 5MB) on the client-side or server-side before sending them to the API to reduce bandwidth usage and improve processing speed.

2. Reliability and Availability

- **Availability:** The web application shall aim for **99% uptime** during operational hours. Since the core logic relies on external APIs, the system must handle API downtimes gracefully by displaying appropriate error messages (e.g., "Service temporarily unavailable, please try again later") instead of crashing.
- **Data Integrity:** The system shall ensure that user profiles and scan history records stored in the database remain consistent and are not corrupted during concurrent access or server restarts.

3. Security Requirements

- **Data Privacy:** User passwords must never be stored in plain text; they shall be hashed using secure algorithms (e.g., SHA-256 or bcrypt) before storage in the database.
- **Secure Communication:** All communication between the client (browser), the web server, and external APIs shall be encrypted using **HTTPS (SSL/TLS)** protocols to prevent data interception.
- **API Key Protection:** The private API keys for Plant.id and Google Gemini shall be stored securely in environment variables on the server and must never be exposed in the client-side source code (HTML/JS).

4. Usability Requirements

- **User Interface (UI):** The interface shall be simple, intuitive, and minimalistic, designed specifically for users with limited technical literacy (e.g., farmers). Critical tasks like "Upload Image" should be achievable in no more than **3 clicks**.
- **Responsiveness:** The web application shall follow **Responsive Web Design** principles. It must automatically adjust its layout, fonts, and images to fit perfectly on various screen sizes, including desktop monitors, laptops, tablets, and smartphones.

5. Portability and Compatibility

- **Browser Compatibility:** The application shall be compatible with all modern web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari (iOS/macOS).
- **Device Independence:** Being a web-based solution, the system shall run on any device (Android, iOS, Windows, Linux) that has a standard web browser and internet connectivity, without requiring any specific hardware installation.

6. Maintainability

- **Code Structure:** The backend code (Python/Flask) shall be modular and well-documented to allow for future updates, such as switching the API provider or migrating the database, with minimal impact on the overall system.

3. Use Cases and Flow of Processes

This section describes the overall **flow of processes** of the CultiKure web application based on the defined functional requirements. The system-level process flow illustrates how different users interact with the system to perform plant disease detection and view cure recommendations.

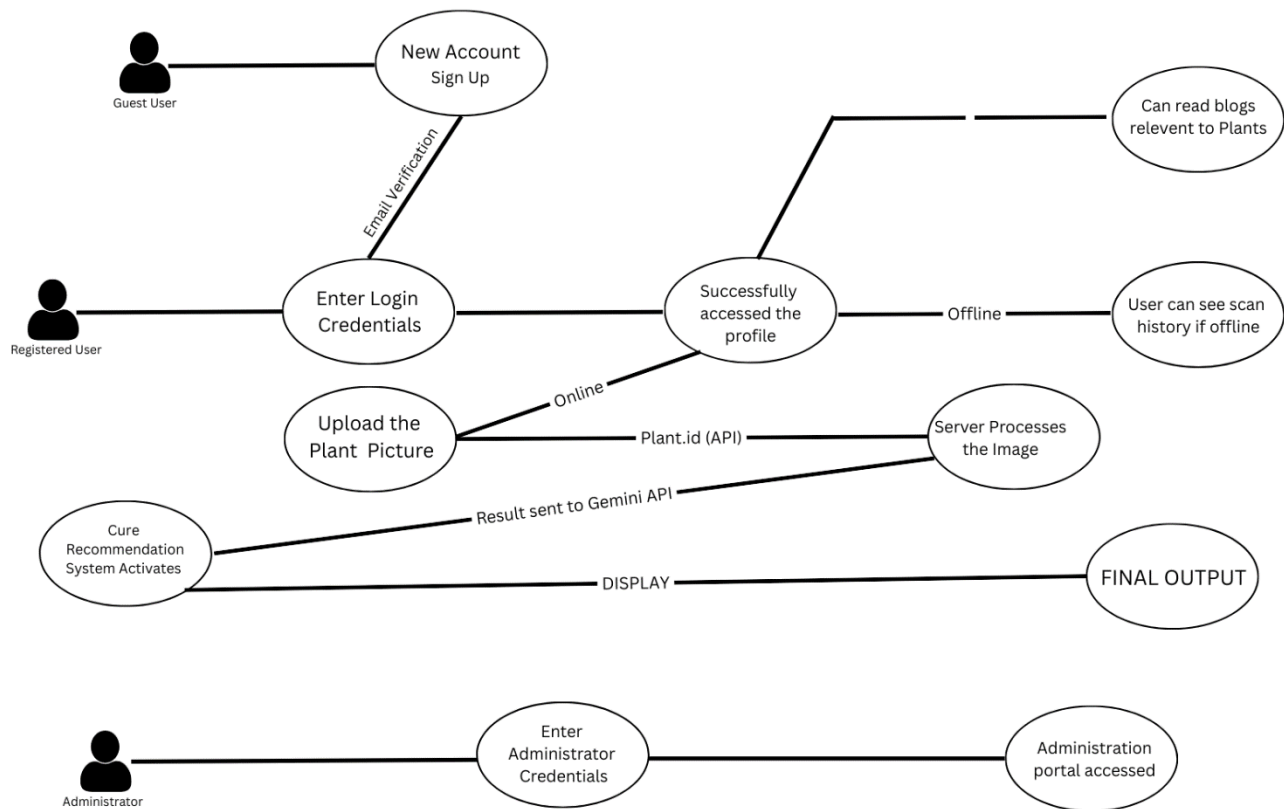


Figure 1: System Level Activity Diagram

3.1. Use Case 1

ID	UC001
Name	Sign Up
Description	This use case describes the process flow for a new user to create a personalized account in the CultiKure system.
Requirement(s)	FR001
Actor(s)	Guest User (Farmer or Gardener)
Precondition	The user must have a stable internet connection.
Postcondition	The user must not already have an account with the provided email address
Basic Flow	1 Access Sign Up: The user navigates to the registration page. 2 Enter Data: The user enters their Full Name, Email Address, Password, and Confirm Password. 3 System Validation: The system validates the inputs: 4 Email must be in a valid format (e.g., name@email.com). 5 Password must be at least 8 characters long. 6 Password and Confirm Password must match exactly. 7 Data Persistence: On successful validation, the system creates a new entry in the user database. 8 Completion: The system displays a success message and redirects the user to the login interface.
Alternative Flow	Already Registered: If the user already has an account, they can click the " Login " link to be redirected to the authentication page.
Exceptions	1. Invalid Inputs: The user provides an incorrect email format or passwords do not match; an error message is displayed 2. Duplicate Account: The email entered is already registered in the system. 3. Network Error: The system is unable to connect to the database due to internet issues.

3.2. Use Case 2

ID	UC002
Name	User Login
Description	This use case describes how a registered user or administrator authenticates themselves to access the system's dashboard and personalized features.
Requirement(s)	FR002
Actor(s)	Registered User, Administrator
Precondition	The user must have a previously registered account in the system database.
Postcondition	The user is granted access to their private dashboard and scan history.
Basic Flow	1. Navigate to Login: User opens the login page. 2. Input Credentials: User enters their registered Email Address and Password. 3. Credential Validation: The system compares the entered email and hashed password with the records in the database. 4. Session Management: On success, the system creates a secure session for the user. 5. Redirection: The user is redirected to the main Dashboard.
Alternative Flow	Forgot Password: User clicks "Forgot Password" to initiate a recovery process (if implemented).
Exceptions	1. Authentication Failure: User enters an incorrect email or password; the system displays an "Invalid Credentials" error. 2. Account Locked: Multiple failed attempts lead to a temporary lock (security measure).

3.3. Use Case 3

ID	UC003
Name	Upload Plant Image for Diagnosis
Description	This use case allows the user to submit a digital image of a diseased plant leaf for real-time AI analysis.
Requirement(s)	FR003
Actor(s)	Registered User
Precondition	1 The user must be logged into the system. 2 The user must have a clear image of the infected plant leaf
Postcondition	The image is successfully processed by the Plant.id API, and the system prepares for the diagnosis display.
Basic Flow	1 Select Source: User clicks the "Upload" button and chooses to either capture a photo via camera or select a file from the gallery 2 File Validation: The system checks if the file is a supported format (JPG/PNG) and within the 5MB size limit. 3 API Transmission: The system sends the image data securely to the Plant.id API for visual processing. 4 Processing Status: The UI displays an "Analyzing..." progress indicator. 5 Identification: The system receives the disease identification result.
Alternative Flow	Retry Upload: If the image is blurry or unidentifiable, the system prompts the user to take a clearer photo.
Exceptions	1 File Too Large: The user tries to upload a file exceeding 5MB. 2 Unsupported Format: User uploads a PDF or non-image file. 3 API Timeout: The external service does not respond within the expected timeframe.

3.4. Use Case 4

ID	UC004
Name	View Diagnosis and Cure Plan
Description	This use case describes the presentation of detailed disease information and AI-generated treatment plans to the user.
Requirement(s)	FR004
Actor(s)	Registered User
Precondition	The system must have successfully received a response from both the Plant.id and Google Gemini APIs.
Postcondition	The diagnostic results are displayed to the user and saved to their scan history.
Basic Flow	Result Generation: The system passes the disease name from Plant.id to the Google Gemini API to fetch contextual cure advice. Display Result Card: The system displays a comprehensive result card containing: - Detected Disease Name. - Detailed Symptoms and Causes. - Actionable Cure Plan (Organic and Chemical remedies). Data Logging: The system automatically saves the scan result, date, and image to the database
Alternative Flow	Print/Share: User chooses to download the recommendation as a summary for offline use.
Exceptions	No Data Found: The AI cannot find a specific cure for a very rare disease; a general advisory message is shown. Latency Issues: Response takes longer than 10 seconds.

3.5. Use Case 5

ID	UC005
Name	View Scan History
Description	This use case allows the user to access a chronological log of all their previous plant disease scans and recommendations.
Requirement(s)	FR005
Actor(s)	Registered User
Precondition	The user must have a history of at least one previous scan saved in the database.
Postcondition	User can review past reports without re-uploading images.
Basic Flow	1. Access History: User clicks on the "History" tab on the main navigation dashboard. 2. Fetch Records: The system retrieves all scan records associated with the user's ID from the SQLite/Firebase database 3. List View: A chronological list is displayed showing thumbnails of plant images, detected disease names, and scan dates. 4. Detail View: User clicks a specific entry to view the full diagnosis and cure details as they appeared originally.
Alternative Flow	Search/Filter: User can search history by date or plant name.
Exceptions	Empty History: If the user has never performed a scan, the system displays a "No records found" message.

4. References

- [1] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, “Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification,” *Computational Intelligence and Neuroscience*, vol. 2016, Article ID 3289801, 2016.
- [2] J. G. A. Barbedo, “Digital image processing techniques for detecting, quantifying and classifying plant diseases,” *SpringerPlus*, vol. 2, no. 660, pp. 1–12, 2013.
- [3] M. Mohanty, D. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” *Frontiers in Plant Science*, vol. 7, Article 1419, 2016.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [5] Plant.id, “Plant.id Health Assessment API Documentation,” 2024. [Online]. Available: <https://web.plant.id/health-assessment>. Accessed: Jan. 2026.
- [6] Google, “Gemini API Documentation,” Google AI, 2024. [Online]. Available: <https://ai.google.dev>. Accessed: Jan. 2026.
- [7] Python Software Foundation, “Flask Web Framework Documentation,” 2024. [Online]. Available: <https://flask.palletsprojects.com>. Accessed: Jan. 2026.
- [8] Firebase, “Cloud Firestore Documentation,” Google, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>. Accessed: Jan. 2026.
- [9] W3C, “HTML5.2 Specification,” World Wide Web Consortium, 2024. [Online]. Available: <https://www.w3.org/TR/html52>. Accessed: Jan. 2026.
- [10] Mozilla Developer Network, “JavaScript Guide,” MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Accessed: Jan. 2026.
- [11] R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Univ. of California, Irvine, USA, 2000.
- [12] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.